Tutorial Sheet 0

Discussion: C-Bootcamp
Translated from Betriebssysteme und Systemsoftware, Prof. Dr-Ing Klaus Wehrle

Task 0.1: Arrays

C arrays represent a continous block of memory which is used to store a fixed number of elements of the same data type. The following

```
int a[10];
```

declares an array containing 10 elements, each one is of type `int`. C does not have any own data type for strings, so strings are represented as an array of `char`, which ends with the character `\0` (strings are null terminated). So the string

```
char s[11];
```

contains 11 characters in total, including the terminating character \0. Strings can be initialized in the variable declaration, for example:

```
char s[11] = "helloworld";
```

When arrays are directly initialized in the declaration, the compiler can deduce and assign its length automatically.

```
char s[] = "helloworld";
```

It is possible to access the elements of an array directly by using its index position. For example:

```
a[2] = 13;
```

It is important to note that the first element has the index 0.

Vigenère-Chiffre is a classical encryption algorithm working like this: The two inputs of are a plaintext message to be encrypted with a plaintext key. First, the message needs to be processed, by changing all non-capital characters to capital characters and removing all blank spaces and special characters. The key should be capitalized and not contain any blank or special characters as well. In addition, the key needs to be repeated until it matches the length of the message. Finally, the letters A to Z are assigned an index from 0 to 25, the digits 0 to 9 are assigned the indexes 26 to 35.

The encryption process involves iterating over each character of both the extended key and the message simultaneously. For each character, we sum up the index of the key and the message. We use the modulo 36 on the sum of the indexes to make sure they remain between 0 and 35.

For example B(1) + E(4) results in F(5). 2(28) + U (20) results in M (12), because we have 48 modulo 36 = 12.

```
Code: BUS2022
Plaintext :   Dieser Text soll verschluesselt werden

Processed :   DIESERTEXTSOLLVERSCHLUESSELTWERDEN
Codeword :    BUS2022BUS2022BUS2022BUS2022BUS202
------------------------------------------------
Ciphertext :  E2WK4JLFHBKEDDWY9K29DVYAK4DLXY954F
```

The receiver of the ciphertext can decrypt it, by writing the code below the ciphertext and subtracting the index values.

Contact: bus@os.rwth-aachen.de

```
Ciphertext :   E2WK4JLFHBKEDDWY9K29DVYAK4DLXY954F
Codeword :     BUS2022BUS2022BUS2022BUS2022BUS202
-----------------------------------------
Plaintext :    DIESERTEXTSOLLVERSCHLUESSELTWERDEN

Plaintext :    Dieser Text soll verschlüsselt werden.
```

Write a program that ask for codeword (with a maximum length of 10 characters) and asks for a text (maximal length of 100 characters). The program then asks the user, whether it should encrypt or decrypt the text. The program should be able to do the processing of the text, decrypt or encrypt based on the user's wish and output the result on the screen.

Task 0.2: Linked Lists

Pointers are well suited for building complex data structures. An example for these is found in **linked lists**. A linked list is a collection of objects which share the same type and are linked to one another by pointers. Every element of the list is represented by a C structure. An example is given in Fig. 1. Here, a list of objects that record students' names and ID numbers is being kept. The list is sorted by ID number in ascending order.

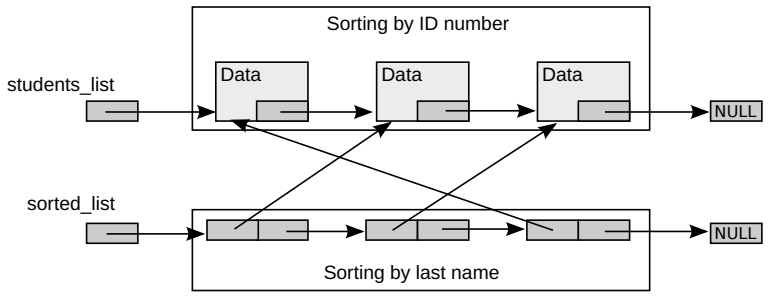

Figure 1: Structure of linked lists.

The following functions are available for access to the linked list:

`is_empty()` tests, whether the list is empty

`enqueue()` adds a new student to the list

`dequeue()` deletes a student from the list

`get_student()` gets a student's data from the list

a) In the Moodle room, together with the exercise sheet, you will find the source code file `lists.c`. Study the source code, compile the program, and test it. You will find out that the functions `enqueue` and `dequeue` have not yet been implemented. Implement both functions using the comments in the source files for guidance. Your code must compile using the following command without warnings.
```
gcc -std=c17 -Wall -Wextra -pedantic -fsanitize=address,undefined -o OUTFILE lists.c
```

b) It should be possible to sort the list by any arbitrary criterion instead of just ID numbers. Write a function `sort_students` that creates a new linear list. The elements of this list should not contain separate instances of the data, but rather link to the original entries of the database (s. Fig. 1). To implement this, use function pointers so that the comparison function used in sorting can be easily adjusted. Pass a function pointer as a parameter to your function that

Contact: bus@os.rwth-aachen.de

generates the new `sorted_list`. Elements that are classified as equal must appear in the order of the original list. Write two comparison functions that can be passed as parameters that sort the list by first, and last name, respectively.

Afterwards, add a usage example at the end of the `main` function: Generate a list sorted by first name and also print it. Use the same output format as `test_dump`.

Note: All of your comparison functions must have the same signature, so keep it generic and always use the following:
`int compare(stud_type const* a, stud_type const* b)`
Usually, a negative `int` return value is used to represent $a < b$, a positive one for $a > b$, and 0 for $a == b$.

What advantages has this form of referencing over a copy of the list?

c) Assume that, for the next exercise, you had to implement a linked list, which would store entries with lecture halls' names and GPS coordinates (as two `doubles`), instead of students' names and ID numbers. How many of the four access functions would have had to be adjusted to support such a list? Why?

Note: You don't have to implement the new list or the access functions, but rather just answer the questions above.

d) Implement functions to store the database from memory permanently in the file system and to read it from there back into memory.

e) Explain what the following macro does:

```
1 #define position(typ, el) ((unsigned long)(&((typ *)0)->el))
```

As an example, the macro can be used as follows: `position(stud_type, next_student)`.

What is the return value? What does it represent?

f) What does this function do wrong? Why is it, in most cases, useless, independently of what it implements?

```
1 int* what_am_i_doing_wrong(void) {
2     int x;
3     /* Something meaningful happens
4      * to the variable here,
5      * e.g. x = 42; */
6     return &x;
7 }
```

Note: It doesn't have anything to do with the fact that the function is using an integer. `int` and `int*` could be replaced by any type and its pointer (`type` and `type*`).

Contact: bus@os.rwth-aachen.de

Annex

```c
#include <stddef.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct stud_type {
    int id;
    char firstname[20];
    char lastname[20];
    struct stud_type *next;
} stud_type;


/* Is the database empty? */
bool is_empty(stud_type const* list) {
    return !list;
}


/* Insert an element
 *
 * Gets a pointer to a pointer, which points to the 1. element of the list
 * Gets ID, first name and last name of the student to be inserted
 *
 * Returns true if successful
 * Returns false to indicate an error
 */
bool enqueue(stud_type** students_list, int id, char const firstname[20], char const
    ↪ lastname[20]) {
    /* Get dynamic memory for the new list entry */

    /* Fill the memory (beware of buffer overflows!) */

    /* Insert the new element into the list */
        /* Is the list empty? */

        /* Sort the student into the right position by ascending ID number */
}

/* Remove an element
 *
 * Gets a pointer to a pointer, which points to the 1. element of the list
 * Gets the ID of the student to be deleted
 *
 * Returns true if successful
 * Returns false to indicate an error
 */
bool dequeue(stud_type** students_list, int id) {
    /* Check edge cases */

    /* Find the student */

    /* Remove the student and free memory */

    /* What must happen if the 1. element of the list is removed? */
    /* What if the ID can't be found in the list? */
    /* ... */

}
```


Contact: bus@os.rwth-aachen.de

```
60
61  /* Read an element
62   *
63   * Gets a pointer to the list type
64   * Gets the ID of the student whose data is to be read
65   *
66   * Writes first and last name into firstname and lastname, respectively
67   */
68  bool get_student(stud_type const* students_list, int id, char firstname[20], char
        ↪ lastname[20]) {
69      /* Search the DB */
70      stud_type const* curr = students_list;
71      while (curr && curr->id < id) {
72          curr = curr->next;
73      }
74
75      if (!curr || curr->id != id) {
76          /* Return value: Error */
77          return false;
78      } else {
79          strncpy(firstname, curr->firstname, 19);
80          firstname[19] = '\0';
81          strncpy(lastname, curr->lastname, 19);
82          lastname[19] = '\0';
83          /* Return value: OK */
84          return true;
85      }
86  }
87
88  static void test_empty(stud_type const* list) {
89      printf(">>> Testing whether the student list is empty ...\n");
90
91      if(is_empty(list)) {
92          printf("    The student list is empty \n");
93      } else {
94          printf("    The student list is not empty \n");
95      }
96  }
97
98  static void test_get(stud_type const* list, int id) {
99      printf(">>> Testing, whether the ID %4i exists ...\n", id);
100
101     char firstname[20];
102     char lastname[20];
103     if(get_student(list, id, firstname, lastname)) {
104         printf("    ID %4i: %s %s\n", id, firstname, lastname);
105     } else {
106         printf("    ID %4i is not known\n", id);
107     }
108 }
109
110 static void test_enqueue(stud_type** list, int id, char const* firstname, char
        ↪ const* lastname) {
111     printf(">>> Inserting the new student into the list: %s %s [%4i] ...\n",
            ↪ firstname, lastname, id);
112     if(enqueue(list, id, firstname, lastname)) {
113         printf("    ID %4i inserted\n", id);
114     } else {
115         printf("    ID %4i couldn't be inserted\n", id);
116     }
117 }
118
```


Contact: bus@os.rwth-aachen.de

```c
static void test_dequeue(stud_type** list, int id) {
    printf(">>> Removing ID %4i ...\n", id);

    if(dequeue(list, id)) {
        printf("    ID %4i removed\n", id);
    } else {
        printf("    ID %4i is not known\n", id);
    }
}

static void test_dump(stud_type const* list) {
    printf(">>> Printing all known students ...\n");
    for(stud_type const* curr = list; curr; curr = curr->next) {
        printf("    ID %4i: %s %s\n", curr->id, curr->firstname, curr->lastname);
    }
}

/* Test all the list functions  */
int main(void) {
    /* Initialize database */
    stud_type* students_list = NULL;

    test_empty(students_list);
    test_enqueue(&students_list, 1234, "Eddard", "Stark");
    test_get(students_list, 1234);
    test_enqueue(&students_list, 5678, "Jon", "Snow");
    test_get(students_list, 1234);
    test_enqueue(&students_list, 9999, "Tyrion", "Lannister");
    test_get(students_list, 1235);
    test_enqueue(&students_list, 1289, "Daenerys", "Targaryen");
    test_dequeue(&students_list, 1234);
    test_empty(students_list);
    test_get(students_list, 5678);
    test_dequeue(&students_list, 9998);
    test_enqueue(&students_list, 1289, "Viserys", "Targaryen");
    test_dequeue(&students_list, 5678);
    test_enqueue(&students_list, 1, "Tywin", "Lannister");
    test_dump(students_list);

    {
        /* Generate list sorted by first name */
        /* Print list */
        /* Delete list */
    }

    {
        /* Generate list sorted by last name */
        /* Print list */
        /* Delete list */
    }

    /* Delete students_list */

    return 0;
}
```


Contact: bus@os.rwth-aachen.de